Hacer parpadear un LED: setup, loop y delay

Lección 6 — 1MAK, SST-INF-01c — Del circuito al primer programa que parpadea de verdad

16/10/2026

1 De dónde partimos: Arduino y sus pines

Arduino es una **placa electrónica**, una cajita con **terminales** (los **pines**, en italiano *pin* o *piedini*), disponibles en unos **conectores** donde se meten los cablecitos. Estos terminales se pueden **programar** con un código: pueden **dar** o **no dar** corriente.

Definición

En el lado de los pines digitales hay **14 terminales** (del 0 al 13) que podemos decidir **encender** o **apagar**. Los dos primeros, el **0** y el **1**, es mejor **no usarlos**, por motivos que veremos con el tiempo (sirven para la comunicación con la computadora). Entonces:

- a la pregunta “¿cuántos terminales digitales tiene Arduino?” la respuesta es **14**;
- a la pregunta “¿cuántos podemos usar sin problemas?” la respuesta es **12**.

Atención

El terminal que hay que conectar **siempre** es **GND**: es la referencia de **0 voltios**, o sea el **terminal negativo** del generador. Cada circuito que sale de un pin tiene que **volver a GND**; si no, el circuito no está cerrado y la corriente no pasa.

2 Un pin es como un generador que se enciende y se apaga

Tomemos el **pin 7**: conectamos un cable, ponemos un **resistor** y **cerramos el circuito** en **GND**. Es el mismo circuito visto en la parte de electrónica: un **generador** y un **resistor**.

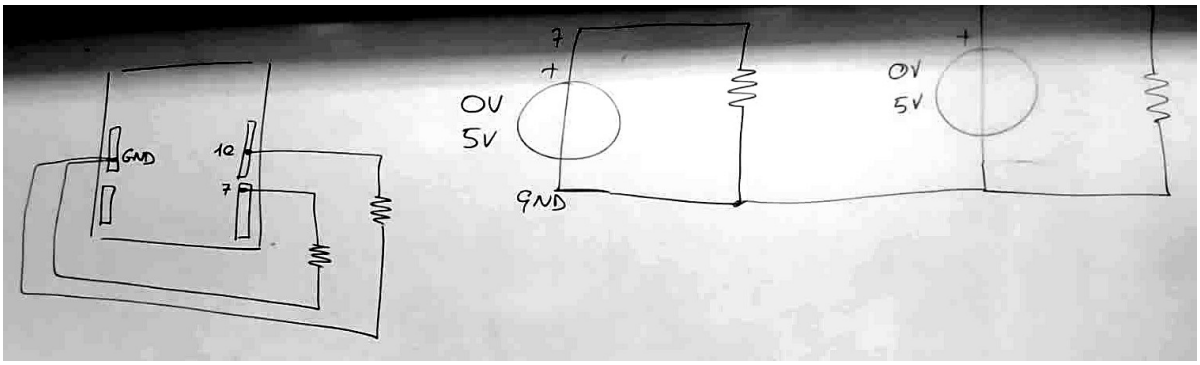


Figura 1: La pizarra. A la izquierda, Arduino como una caja: del pin 7 y del pin 12 salen dos circuitos, cada uno con su resistor, que vuelven los dos a GND. A la derecha, el esquema equivalente: cada pin es un generador que puede dar 0 V o 5 V, y los dos generadores tienen el GND en común.

Definición

Con el programa podemos decirle a este "generador" que produzca **0 voltios** o **5 voltios**:

- con **0 V** no hay ninguna **fuerza** que empuje los electrones: **no pasa corriente**, y una lámpara conectada queda **apagada**;
- con **5 V** hay una **fuerza** que empuja los electrones: los electrones empiezan a **moverse**, pasa **corriente**, y la lámpara se **enciende**.

Entonces, desde el programa podemos decir: **deja pasar la corriente** o **no la dejes pasar**.

Con 12 pines utilizables se pueden hacer **todos los circuitos** que hagan falta. Por ejemplo, en el **pin 12** se hace otro circuito, con otro resistor, que también vuelve a **GND**: es como tener un **segundo generador**, con el GND **en común** con el primero. También este puede ir de 0 a 5 V, y podemos decidir qué circuito encender **según lo que necesitemos**.

Ejemplo

El resistor representa "cualquier cosa". En el esquema, el resistor representa **cualquier dispositivo** que queramos controlar:

- un **resistor** de verdad, por ejemplo el de una estufita que **calienta**;
- una **lámpara**, para un sistema de encendido de las luces de una casa;
- un **motor**, una **bomba** que lleva el agua en una instalación, un **ventilador** que enfría un dispositivo demasiado caliente (aquí con cierta libertad: para controlarlos hace falta un circuito un poco más complejo);
- un **LED**, una pequeña lámpara que podemos hacer **encender** y **parpadear** según lo que queramos comunicar.

Nota

Los viejos **carteles luminosos** de las paradas de autobús (hoy muchas veces reemplazados por monitores) estaban hechos de **muchas lamparitas**, o sea muchos LED: encendiéndolos de la manera correcta se **forma el texto**. Quien decide qué LED encender, cómo, dónde y en qué momento es un microcontrolador como el de Arduino. Con Arduino se puede hacer de todo: depende de la **imaginación** y de la **práctica**.

3 Primero la teoría, después la práctica

Nota

Nuestro curso apunta mucho a la **práctica**: lo importante es **saber hacer** las cosas. Pero sin **teoría** la práctica cuesta mucho. Es como poner a un monito delante de una máquina de escribir: aunque apriete teclas durante horas, no se vuelve poeta si nadie le explica cómo se escribe un texto.

4 Las palabras del primer programa

En la pizarra se juntaron las **palabras** vistas hasta ahora, que sirven para escribir el primer programa. Están todas en las guías anteriores, sobre todo en la A05:

Palabra	Para qué sirve
`void`	palabra que va antes del nombre de un bloque de código
`setup`	bloque que se ejecuta una sola vez, al principio
`loop`	bloque que se repite infinitamente
`pinMode`	decide si un pin es una entrada o una salida
`INPUT` / `OUTPUT`	entrada (oreja) / salida (boca)
`digitalWrite`	enciende o apaga un pin de salida
`HIGH` / `LOW`	encendido (5 V) / apagado (0 V)
`delay`	detiene la máquina durante un tiempo

Atención

Las guías hay que estudiarlas. Estas palabras ya estaban en la guía publicada después de la clase anterior: en clase se da por hecho que las guías se **leyeron**, si es posible varias veces. Para el **martes** estos comandos hay que saberlos.

5 El bloque setup

Definición

Todo programa de Arduino tiene **zonas bien definidas**. La primera es el bloque `void setup()`:

- hay **uno solo**;
- se ejecuta **una sola vez**, cada vez que se enciende la máquina o se le carga el programa;
- está encerrado entre dos **llaves**: la que se abre { marca el comienzo, la que se cierra } el final del bloque;
- adentro van **solo las configuraciones iniciales** de la máquina, por ejemplo la dirección de los pines.

5.1 pinMode: una boca en el pin 10

Ejemplo

La boca. Si quieres decirle algo a la clase, ¿qué usas? ¿La nariz? ¿Los ojos? No: la **boca**. La boca es un "periférico" del cuerpo que sirve para **emitir** sonidos y comunicar algo a los demás: del cuerpo **sale** algo. Para escuchar, en cambio, se usan las **orejas**: algo **entra**.

Queremos crear una "boca" en el **pin 10**, para que Arduino pueda decir algo hacia afuera. Tenemos que decirle a Arduino que el pin 10 es una **salida**:

```
pinMode(10, OUTPUT);
```

Entre los paréntesis: el **número del pin** (10, pero puede ser 3, 7, 13, el que necesitemos), una **coma**, y `OUTPUT`. Si en cambio quisiéramos una "oreja", o sea un pin que **escucha** lo que tienen para decir otros dispositivos externos, por ejemplo en el pin 5, escribiríamos:

```
pinMode(5, INPUT);
```

Nota

Por ahora usaremos solo las **salidas** (la boca): para usar bien las entradas (las orejas) hay que aprender cosas nuevas.

5.2 Las reglas de escritura

Atención

- Cada instrucción en C termina con **punto y coma** ;. Siempre. Si te lo olvidas, la máquina marca un **error**.
- El C distingue **mayúsculas y minúsculas**: los comandos se escriben **exactamente** así. `pinMode` tiene la **M mayúscula**; `INPUT` y `OUTPUT` van **todo en mayúsculas**. Escribir `pinmode` da error.
- "Me equivoqué solo en una mayúscula" no alcanza: el programa está **mal**. La programación **no deja espacio** al **error de sintaxis** (o sea de escritura): la máquina no dice "bueno, igual entendí".

6 El bloque loop y digitalWrite

Definición

Después del setup, la máquina empieza a **repetir infinitamente** otro bloque: **`void loop()`**. Entre sus llaves se escribe el código que hay que **repetir continuamente**. Si dejas la máquina encendida mil años, lo repite mil años: no se aburre, no se cansa, y mientras tenga energía sigue.

Definición

Para **encender** o **apagar** un circuito, o sea un pin, se usa **`digitalWrite`** (con la **W mayúscula**). Entre los paréntesis: el **pin**, una coma, y **`HIGH`** (encendido) o **`LOW`** (apagado). Y siempre el punto y coma.

```
digitalWrite(10, HIGH);   // enciende el circuito del pin 10
digitalWrite(10, LOW);    // lo apaga
```

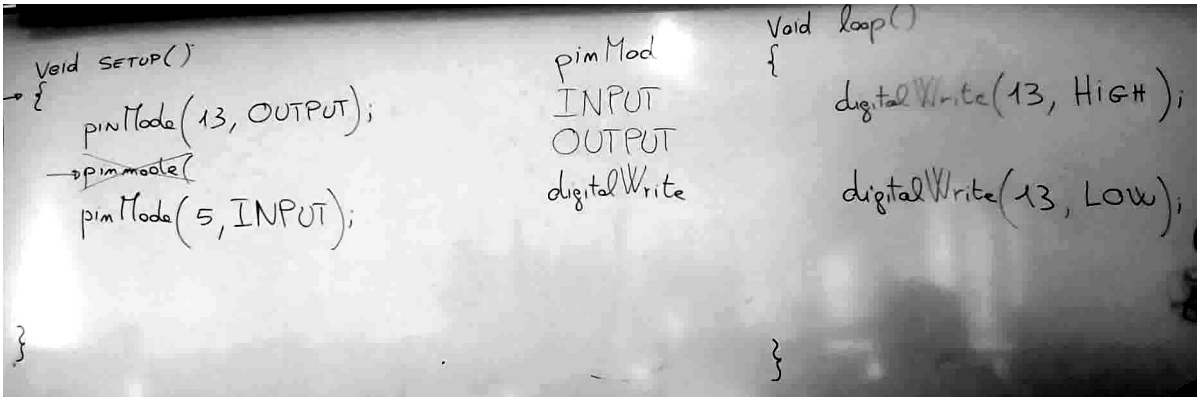


Figura 2: El código en la pizarra: a la izquierda el setup con `pinMode(13, OUTPUT);` y `pinMode(5, INPUT);` (la palabra `pimmode(` está tachada: escrita así es un error); en el centro las palabras que hay que escribir con las mayúsculas correctas; a la derecha el loop con `digitalWrite(13, HIGH);` y `digitalWrite(13, LOW);`.

7 Probamos en Tinkercad

En el simulador, en el pin 10 no hay nada conectado. Para no complicarnos usamos el **pin 13**, que ya tiene una **lamparita** (el LED **L**) conectada en la placa.

7.1 Primer intento: solo HIGH

```
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
}
```

Nota

Conviene escribir enseguida **las dos llaves**, la que abre y la que cierra, antes de llenar el bloque: si falta una, el programa da **error** y no funciona nada.

Al iniciar la simulación la lamparita se **enciende**. El programa está diciendo continuamente "enciende el LED, enciende el LED, enciende el LED…": la máquina no se aburre y no contesta "ya me lo dijiste".

7.2 Segundo intento: HIGH y después LOW

```
void loop()
{
  digitalWrite(13, HIGH);
  digitalWrite(13, LOW);
}
```

¿El LED está encendido o apagado? A la vista parece **siempre encendido** (o casi). En realidad está **parpadeando**, pero **demasiado rápido** para que el ojo lo note. Con un **instrumento de medida** se vería que se enciende y se apaga continuamente.

Definición

Arduino tiene un **reloj** (*clock*) interno de **16 MHz**: hace unas **16 millones** de operaciones **por segundo**. Encender y apagar un pin no es una sola operación, hacen falta más, pero igual el LED se enciende y se apaga **cientos de miles** de veces por segundo: mientras nosotros decimos "uno", él ya se encendió y se apagó muchísimas veces.

Nota

Aclaración respecto a la clase: en el aula se dio, a grandes rasgos, una estimación de unos 500.000 parpadeos por segundo. Con los comandos de Arduino (`digitalWrite` tarda algunos microsegundos) el valor real es más bajo, del orden de algunos cientos de miles de cambios de estado por segundo; la idea no cambia: es **demasiado rápido** para el ojo.

Ejemplo

Los FPS de los videojuegos. Los **FPS** (*frames per second*, cuadros por segundo) dicen cuántas veces por segundo la computadora crea y manda al monitor una **imagen nueva**. Para mostrar un personaje que se mueve, muestra una foto, después otra con el pie un poco más adelante, después otra más. A 120 FPS, en un segundo pasan 120 fotos, cada una muy poco distinta de la anterior, y el movimiento parece **fluido**. Por encima de los **25 cuadros por segundo** el ojo ya no nota las imágenes sueltas. En los juegos más rápidos hacen falta muchos más FPS, para no ver saltos.
Un monitor de 1920×1080 tiene unas **2 millones** de "lamparitas" (los píxeles) que encender y apagar con criterio; aquí tenemos **una sola**, y Arduino la enciende y la apaga rapidísimo. Si en cambio le pidiéramos dibujar una imagen de 2 millones de píxeles, iría lentísimo: es una máquina rápida para operaciones **muy simples**.

8 delay: démosle tiempo al ojo

Definición

Para ver el parpadeo hay que decirle a la máquina que **espere** entre el encendido y el apagado: se hace con **`delay`**. El comando **bloquea** Arduino (lo "congela", lo hace "laguear" a propósito) durante el número de **milisegundos** escrito entre los paréntesis. `delay(1000)` quiere decir mil milisegundos, o sea **un segundo**.

Atención

El `delay` hace falta **dos veces**: después del `HIGH` y también después del `LOW`. Con un solo `delay` después del encendido, el LED estaría encendido un segundo, después se apagaría y enseguida, sin espera, el loop volvería a empezar encendiéndolo otra vez: el apagado **no se vería**.

```
void loop()
{
  digitalWrite(13, HIGH);
  delay(1000);    // encendido un segundo
  digitalWrite(13, LOW);
  delay(1000);    // apagado un segundo
}
```

Ahora la lamparita se ve **parpadear**: encendida un segundo, apagada un segundo.

8.1 Más rápido, más rápido…

Valor del delay	Qué se ve en el simulador
1000	parpadeo lento, un segundo encendido y un segundo apagado
500	parpadeo más rápido
200	más rápido todavía
100	todavía se ve parpadear
60	se ve “algo”, a saltos
50	casi no se ve más: no llega a apagarse que ya tiene que volver a encenderse

Nota

El límite del simulador. Tinkercad simula una máquina **sobre otra máquina**: es mucho **más lento** que un Arduino de verdad, y no se puede apreciar el parpadeo como en la realidad. Con la placa real se logra ver el parpadeo hasta valores de unos **10–15 milisegundos**. Cuando lo hagamos en vivo, la diferencia se va a ver bien.

Ejemplo

Por qué a veces parece que cambia el brillo. Con valores bajos, el LED del simulador puede parecer no ya encendido y apagado, sino **más o menos brillante**. No es así: es una **interferencia** entre la frecuencia con la que se actualiza el **monitor** (sus FPS) y la frecuencia del parpadeo. Es el mismo **efecto óptico** de cuando, en el auto, se pasa a otro auto y parece que sus **ruedas giran al revés**: el cerebro “saca fotos” continuamente, pero si la rueda gira demasiado rápido se pierden momentos, y en la secuencia la rueda parece ir hacia atrás en vez de hacia adelante. Con la placa real, el parpadeo se ve de otra manera.

9 Qué recordar de esta lección

- Arduino tiene **14 pines digitales**; el **0** y el **1** no se usan, entonces quedan **12**. Cada circuito sale de un pin y vuelve a **GND**.
- Un pin es como un **generador** que el programa pone en **0 V** (apagado) o en **5 V** (encendido); en lugar del resistor puede haber cualquier dispositivo.
- **`void setup()`** se ejecuta una sola vez y tiene las configuraciones (ej. `pinMode(13, OUTPUT);`); **`void loop()`** se repite infinitamente.
- Punto y coma al final de cada instrucción; mayúsculas exactas: `pinMode`, `digitalWrite`,

`INPUT`, `OUTPUT`, `HIGH`, `LOW`. Un error de escritura es un error.

- Sin `delay` el LED parpadea **demasiado rápido** para el ojo (reloj de 16 MHz). Hace falta un **`delay`** después del `HIGH` y otro después del `LOW`; el número va en **milisegundos**.
- El simulador es más lento que la placa real; con valores bajos puede aparecer una **interferencia** con el monitor.

10 Glosario: palabras en italiano que se usan en clase

Italiano	Español
scheda	placa
piedino, pin	pin, terminal
generatore	generador
resistore, resistenza	resistor, resistencia
lampadina	lámpara, lamparita
acceso / spento	encendido / apagado
accendere / spegnere	encender / apagar
lampeggiare, lampeggio	parpadear, parpadeo
ingresso / uscita	entrada / salida
bocca / orecchie	boca / orejas
graffe	llaves
tonde	paréntesis
virgola / punto e virgola	coma / punto y coma
maiuscola / minuscola	mayúscula / minúscula
errore	error
millisecondi	milisegundos
Avvia simulazione	Iniciar simulación
dispensa	guía (apuntes del profesor)